Lin Zhu

Professor. Easton

ISOM-674-4101

13 December 2025

CTR Prediction – Project Report

1. Introduction and Overall Workflow

Click-through rate (CTR) prediction is a fundamental component of modern online advertising systems. In real-world ad platforms, predicted click probabilities are used not only to rank advertisements but also to determine pricing, bidding strategies, and budget allocation. As a result, even small improvements in probability estimation can lead to meaningful downstream business impact.

The objective of this project is to predict the probability that an online advertisement will be clicked using a large-scale and highly imbalanced dataset containing approximately 32 million observations. The target variable, click, is binary, with an overall click-through rate (CTR) of approximately 15%. This level of class imbalance makes the problem particularly challenging, as naive models can easily achieve high accuracy by predicting the majority class while producing poorly calibrated probabilities.

To address this issue, model performance is evaluated using log-loss rather than accuracy. Log-loss directly penalizes incorrect and overconfident probability estimates, encouraging models to produce calibrated predictions that reflect true uncertainty. This evaluation metric closely aligns with the objectives of real-world CTR optimization systems, where probability quality is more important than discrete classification outcomes.

Given the scale and complexity of the data, our modeling strategy was designed to balance statistical soundness, computational feasibility, robustness against data leakage, and practical predictive performance. Due to hardware and storage limitations on the EMR cluster, training complex models on the full dataset was infeasible. As a result, we constructed a scalable pipeline based on careful sampling, efficient feature engineering, and progressive model selection.

Our final workflow consists of the following steps: stratified sampling and data reduction, feature engineering for high-cardinality categorical variables, train–validation splitting with strict leakage control, baseline modeling using logistic regression, exploration of tree-based models, final model selection using CatBoost, and large-scale test-time inference. Each stage of this workflow was designed to address a specific challenge inherent in large-scale CTR prediction.

2. Data Processing

2.1 Challenges

The original dataset contains approximately 32 million rows with more than 40 categorical variables. Many features, such as site_id, app_id, and anonymized variables C14–C21, exhibit extremely high cardinality with tens of thousands of unique values. These variables typically represent identifiers or contextual information that is common in advertising data but difficult to model using traditional techniques.

One-hot encoding, while conceptually simple, is infeasible in this setting due to the resulting dimensional explosion. Even when sparse representations are used, the memory and computational overhead associated with extremely high-dimensional feature spaces becomes prohibitive at scale.

In addition, Spark-based tree models on the full dataset required extensive shuffle operations and disk spill, exceeding available EMR node storage. These issues significantly increased training time and led to unstable execution. Together, these constraints motivated the use of sampling and alternative modeling approaches that could better balance performance and feasibility.

2.2 Sampling Strategy

To ensure computational feasibility while preserving statistical representativeness, we adopted a two-stage stratified sampling strategy. The original data spans nine consecutive days, and daily stratified sampling was used to preserve temporal structure and daily CTR patterns. This design ensures that the sampled data reflects both the marginal class distribution and the temporal dynamics present in the full dataset.

The target sample size was approximately 1 million observations. This sample size was chosen as a compromise between computational tractability and statistical power. At this scale, models could be trained and tuned efficiently while still capturing meaningful signal from the data.

This approach preserved the overall CTR distribution (approximately 0.15–0.18), reduced memory and disk usage, and enabled stable model training. The sampled dataset was saved to Amazon S3 in Parquet format and reused for all subsequent experiments, avoiding repeated reads of the original 32-million-row CSV file. A larger 3-million-row sample was also saved for robustness checks, although the 1-million-row sample was used for most experiments.

3. Feature Engineering

3.1 Hashing-Based Feature Engineering

Given the extremely high cardinality of categorical features, we implemented a hashing-based feature engineering pipeline. Low-cardinality categorical variables were processed using StringIndexer, while high-cardinality variables were encoded using FeatureHasher. This approach maps categorical values into a fixed-dimensional feature space, enabling scalable modeling without explicitly enumerating all possible categories.

All feature vectors were combined using a VectorAssembler to produce a unified representation suitable for downstream models. Hashing-based approaches are widely used in industrial CTR systems because they provide a practical balance between expressiveness and scalability.

To prevent data leakage, the entire feature engineering pipeline was fitted exclusively on the training set, and the learned transformations were applied to the validation set. This ensured that no information from the validation data influenced feature construction or model training.

3.2 Choice of Hash Dimension

A hash dimension of 2^16 (65,536) was selected to balance collision risk, memory efficiency, and training speed. While hash collisions are unavoidable in this approach, moderate collision rates are generally acceptable in CTR prediction tasks, particularly when the number of observations is large and the signal-to-noise ratio is relatively low.

This choice reflects standard practice in large-scale CTR prediction tasks and provided a practical trade-off between representational capacity and computational cost. Increasing the hash dimension further would have increased memory usage and training time without clear empirical benefits under the given constraints.

4. Train–Validation Split and Leakage Prevention

All sampling was completed prior to model fitting to ensure a clean separation between data preparation and model training. The sampled dataset was split into training (80%) and validation (20%) sets using label stratification to preserve the click-through rate distribution.

All feature transformations, including indexing, hashing, and vector assembly, were fitted solely on the training data. The fitted pipelines were then applied to the validation data without

refitting, ensuring that no information from the validation set leaked into training. This design choice is particularly important in CTR prediction tasks, where subtle forms of leakage can lead to overly optimistic performance estimates.

5. Models and Rationale

5.1 Logistic Regression

Logistic regression (LR) was used as a baseline model due to its strong performance on sparse, high-dimensional data, efficient training, and well-calibrated probability outputs. LR is a standard benchmark for CTR prediction and is widely used in industry due to its simplicity, interpretability, and scalability.

Hyperparameter tuning focused on the L2 regularization parameter while holding other parameters constant (maxIter = 80, elasticNetParam = 0). The regularization parameter was varied over {1e-5, 1e-4, 5e-4, 1e-3}. Models were evaluated using validation log-loss as the primary metric and AUC as a secondary diagnostic.

The best-performing LR model achieved a validation log-loss of approximately 0.412 with an AUC in the range of 0.72–0.73. These results indicate strong ranking performance and stable generalization under sparse hashed features, confirming that LR captures meaningful linear signals in the data.

5.2 Tree-Based Models

Random Forest models were explored to capture potential non-linear interactions among features that are not accessible to linear models. Tree-based methods are, in principle, well-suited to modeling complex interactions and heterogeneous effects.

Due to computational constraints, lightweight configurations were used, including shallow tree depth, a limited number of estimators, and higher minimum node sizes. Although Random Forest models achieved a validation log-loss of approximately 0.45, their AUC remained close to 0.52, indicating weak ranking capability and poorly calibrated probabilities.

These results suggest that, under the given constraints, Spark-based tree models were unable to effectively exploit the available signal and were therefore not selected as the final approach.

6. CatBoost Model for CTR Prediction

6.1 Motivation

The limitations of logistic regression and Spark-based tree models motivated the use of CatBoost as the final modeling approach. CatBoost natively handles high-cardinality categorical features using ordered target statistics, which reduces information loss and mitigates target leakage.

In addition, CatBoost captures non-linear feature interactions while remaining computationally efficient for million-scale datasets. These properties make it particularly well-suited for CTR prediction tasks involving complex categorical data.

6.2 Data Preparation

The CatBoost model was trained on a sampled Parquet dataset containing approximately 1 million observations and 24 columns. The target variable was defined as $\texttt{click} \in \{0, 1\}$. Identifier columns and the target variable were removed from the feature matrix.

A numerical feature, hour_of_day, was extracted from the timestamp-style hour variable by taking the last two digits. The final feature set consisted of 23 features, including 22 categorical variables and one numerical variable. Categorical feature indices were explicitly provided to CatBoost to enable native categorical handling.

6.3 Class Imbalance Handling

CTR prediction is inherently imbalanced, with a positive rate of approximately 15%. To address this, class weights were computed from the training data and incorporated into the CatBoost training process. This adjustment penalizes misclassification of the minority class more heavily and improves probability calibration.

As a baseline reference, a constant-probability predictor using the empirical CTR achieved a validation log-loss of 0.4565. This baseline provides a useful lower bound for evaluating model improvements.

6.4 Model Specification and Tuning

The CatBoost classifier was trained using log-loss as both the training objective and evaluation metric. Early stopping was enabled with a patience of 100 iterations, and a fixed random seed was used to ensure reproducibility.

Hyperparameter tuning focused on tree depth, learning rate, and L2 regularization. The best-performing configuration used a depth of 6, a learning rate of 0.05, and an L2 leaf regularization parameter of 3. Early stopping selected the optimal number of boosting iterations automatically.

6.5 Results and Model Selection

Validation log-loss consistently decreased during training, and the best performance was achieved at iteration 1998. The final CatBoost model achieved a validation log-loss of 0.39297, representing a substantial improvement over both the constant baseline and logistic regression.

6.6 Test-Time Inference

The trained CatBoost model was used to generate predictions for the full test dataset containing over 13 million observations. Due to the dataset size, inference was performed in fixed-size chunks to ensure memory safety and to preserve the original row order required by the submission format.

Predicted probabilities ranged from approximately 0.0028 to 0.9769, indicating non-degenerate and well-calibrated outputs suitable for downstream evaluation.

7. Discussion and Limitations

Several limitations of the proposed approach should be noted. First, model training relied on a stratified subsample rather than the full dataset due to computational constraints. While the sampling strategy preserved key distributional properties, additional performance gains may be achievable with larger training samples.

Second, hyperparameter tuning for CatBoost was limited to a small grid for tractability. More extensive tuning or alternative optimization strategies could further improve performance but were beyond the available computational budget.

Finally, while CatBoost provides strong predictive accuracy, it offers limited interpretability compared to linear models. In practical applications, additional post-hoc analysis may be required to improve model transparency and stakeholder trust.

8. Conclusion

This project developed a scalable CTR prediction pipeline for a large, high-cardinality, and highly imbalanced dataset. Through careful sampling, leakage-aware feature engineering, and systematic model comparison, CatBoost emerged as the most effective modeling approach.

The final model achieved a substantial reduction in validation log-loss relative to baseline predictors and linear models, while remaining computationally feasible at scale. Overall, this

work demonstrates the importance of aligning modeling choices with data characteristics and operational constraints in large-scale probabilistic prediction tasks.

Appendix:

Appendix A: Code Organization and Reproducibility

This appendix provides a brief overview of the code files used in this project. All experiments were conducted using Python and Spark-based notebooks, with model training and evaluation performed on sampled datasets stored in Amazon S3. The code was structured to ensure reproducibility, prevent data leakage, and allow incremental experimentation under realistic computational constraints.

Data Processing & LR & RF.ipynb:

This notebook contains data loading, stratified sampling, and feature engineering using StringIndexer, FeatureHasher, and VectorAssembler. It also includes baseline model training using Logistic Regression and Random Forest, along with validation log-loss and AUC evaluation.

CatBoost Trial 1.ipynb:

This notebook documents initial CatBoost model training on the sampled dataset. It specifies categorical feature indices, applies class weights to address class imbalance, and uses early stopping based on validation log-loss.

CatBoost Parameter Tuning.ipynb:

This notebook focuses on hyperparameter tuning for CatBoost, including exploration of tree depth, learning rate, and L2 regularization. The final model configuration was selected based on validation log-loss performance.

Throughout all experiments, feature transformations were fitted exclusively on training data, and validation data was used only for performance assessment. Test-time inference was conducted after final model selection, with predictions generated in fixed-size batches to ensure memory safety and preserve row order.